

Proyecto 1 — Base de datos distribuida a pequeña escala

Instituto Tecnológico de Costa Rica · TI-4601 Bases de Datos Avanzadas

Dominio	Opción A — Banca / billetera regional
Regiones	cr-sj (primaria), cr-limon, us-east
Motor	CockroachDB v24.3.0, 3 nodos (clúster del Lab 1), un nodo por región
Baseline de comparación	PostgreSQL 16, un nodo
Cliente	Python 3 + psycopg3, siempre dentro del contenedor del curso
Repositorio	rama Development; evidencia versionada en evidence/p1/
Integrante	Carné Usuario de git
Christopher Rodriguez C.	2022040771 ChrisRC7
Fabrizio Mena M.	2019042722 fabrimena
Alisson Redondo M.	2021510425 AlissonRM19

Resumen

Se diseñó, implementó y midió una base de datos bancaria fragmentada horizontalmente por región sobre un clúster CockroachDB de tres nodos, uno por región simulada.

- **Diseño (E1).** cliente se fragmenta por region (fragmentación primaria); cuenta y movimiento heredan esa partición por fragmentación derivada, forzada con claves foráneas compuestas. moneda se replica completa (GLOBAL). Se verifican completitud, reconstrucción y disyunción con consultas SQL sobre los datos cargados.
- **Implementación (E2).** Un solo script (p1/setup.sh) levanta el clúster, crea las regiones, el esquema con LOCALITY, carga datos deterministas (huella ad995e29a16440f7) y verifica 10 propiedades. Reproducido en la máquina de otro integrante.
- **Mediciones (E3).** n = 100 por caso, warm-up descartado, p50 y p99. La lectura remota cuesta ~3× en el p99; las dos escrituras cuestan lo mismo, y el informe explica por qué (un nodo por región ⇒ el quórum

siempre cruza regiones).

- **Falla (E4).** Tres corridas deteniendo el nodo que tenía el lease: **RTO 4,6 / 6,5 / 5,2 s** y **RPO 0** para escrituras confirmadas, demostrado con una serie de folios sin huecos. Además se midió la caída de una región completa y su efecto sobre las filas REGIONAL BY ROW.
- **Crítica (E5).** Contra PostgreSQL de un nodo en la misma máquina, el clúster es 4,5–11× más lento y usa ~21× la memoria. Conclusión: **justificado solo por la regla de residencia.**

Convención de este informe. Cada afirmación numérica cita el archivo de `evidence/p1/` que la respalda, y todos esos archivos los genera un script del repositorio. Los límites del montaje (regiones lógicas, sin latencia inyectada, subreplicación de REGIONAL BY ROW con un nodo por región) se declaran en cada entregable en lugar de omitirse.

Requisitos y regiones

El dominio es una billetera regional con tres regiones: `cr-sj` (sede central, San José), `cr-limon` (sucursal regional) y `us-east` (clientes en Estados Unidos, remesas). Las entidades del enunciado son `cliente`, `cuenta` y `movimiento`, más un catálogo pequeño (`moneda`) que sirve de candidato a tabla replicada.

Regla de residencia (enunciado). La PII del cliente —nombre y documento— solo debe existir en su región de apertura. La región de apertura es donde el cliente firmó el contrato y no cambia: es la región hogar de su fila de `cliente`, de sus cuentas y de sus movimientos.

Operaciones y frecuencias estimadas están en el entregable 1, sección 1. Las dos conclusiones que gobiernan todo el diseño: ~95 % de las operaciones tocan una sola región, y la única escritura frecuente que cruza regiones es la transferencia entre clientes de regiones distintas (O5), que es justamente la "escritura que cruza región" medida en E3.

Perfil de ejecución. Tres contenedores CockroachDB en una sola máquina, con `--cache=256MiB --max-sql-memory=256MiB`, sin latencia inyectada entre regiones. Las regiones son lógicas: las cifras de E3 no representan una WAN.

E1 — Diseño de fragmentación y asignación (25 %)

Dominio: Opción A, banca / billetera regional · **Regiones:** cr-sj, cr-limon, us-east · **Motor:** CockroachDB ×3 (clúster del Lab 1)

1. Requisitos y regiones

Región	En el dominio	Nodo
cr-sj	Sede central, San José; concentra la mayoría de clientes	crdb-1 (gateway de E3)
cr-limon	Sucursal regional, Limón	crdb-2
us-east	Clientes residentes en EE. UU. (remesas)	crdb-3

La **región de apertura** es donde el cliente firmó su contrato y no cambia. Es la región hogar de su fila de cliente, de sus cuentas y de sus movimientos. **Regla de residencia (enunciado):** la PII (nombre, documento) solo existe en la región de apertura.

Frecuencias estimadas (supuestos de diseño, por cada 1 000 operaciones de una región en un día):

#	Operación	Tipo	Frec. %	Local / cruza región
O1	Consultar saldo	lectura	400	local
O2	Últimos <i>N</i> movimientos de una cuenta	lectura	250	local
O3	Depósito o retiro	escritura	150	local
O4	Transferencia, misma región	escritura	100	local
O5	Transferencia, regiones distintas	escritura	30	cruza región
O6	Identificar cliente por documento	lectura	40	local; la PII no sale
O7	Consultar tasa / catálogo de monedas	lectura	300 (dentro de O3–O5)	desde todas las regiones

#	Operación	Tipo	Frec. % Local / cruza región
O8	Alta de cliente y cuenta	escritura	5 local (unicidad de documento cruza)
O9	Reporte consolidado	lectura	< 1 cruza todas (batch)

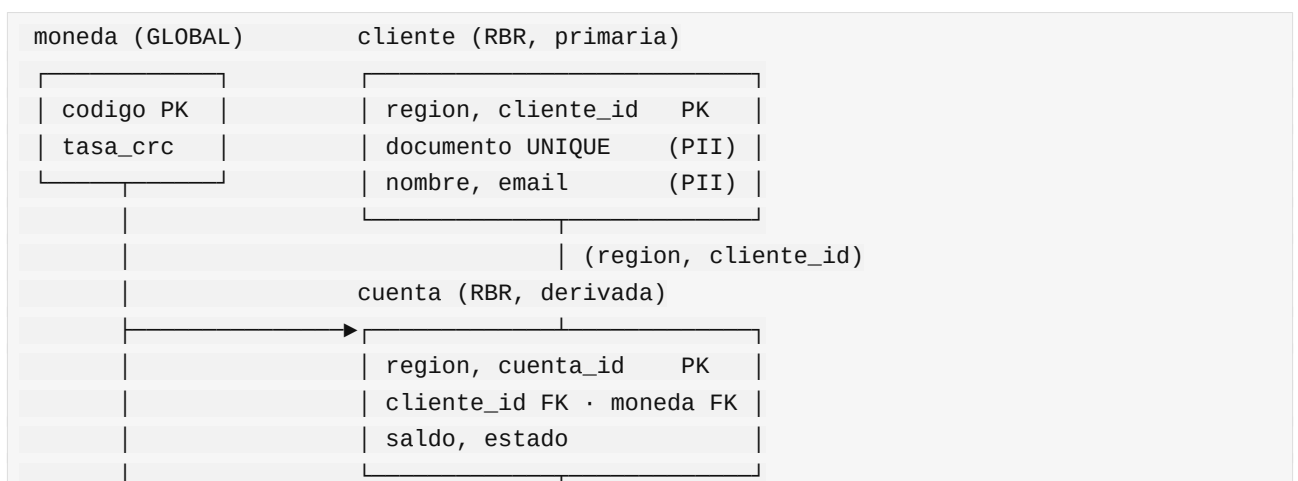
Conclusiones que gobiernan el diseño: **~95 % de las operaciones tocan solo su región** → fragmentar por región.
O5 es la única escritura frecuente que cruza región → es la "escritura que cruza región" de E3; O4 es la "escritura local". **moneda se lee desde todas las regiones y se escribe ~1 vez al día** → candidata a GLOBAL.

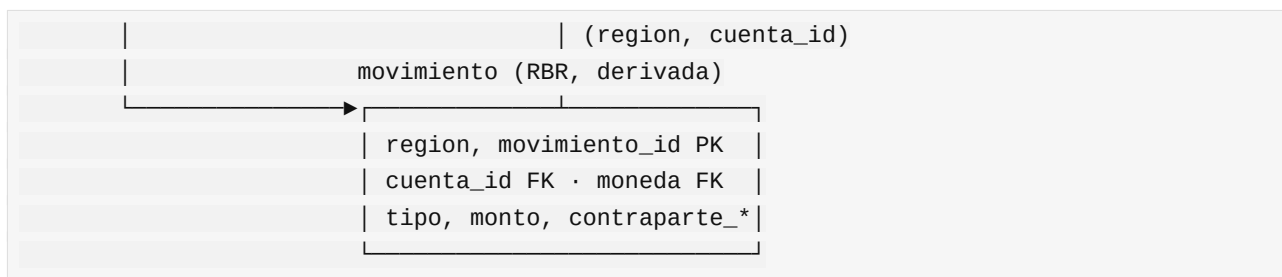
Volumen supuesto para el seed (E2 puede ajustarlo justificándolo): 3 000 clientes (50 / 30 / 20 % por región), ~4 500 cuentas, ~90 000 movimientos, 3 monedas (CRC, USD, EUR).

2. Esquema lógico

PK en **negrita**; → FK; (PII) datos personales.

Relación	Columnas	Localidad
moneda	codigo CHAR(3), nombre, tasa_crc DECIMAL > 0, actualizado_en	GLOBAL
cliente	region , cliente_id UUID, documento UNIQUE (PII), nombre (PII), email (PII), creado_en	RBR, primaria
cuenta	region , cuenta_id UUID, cliente_id, moneda → moneda, saldo ≥ 0, estado, abierta_en; FK (region, cliente_id) → cliente	RBR, derivada
movimiento	region , movimiento_id UUID, cuenta_id, tipo, monto > 0, moneda → moneda, saldo_resultante, contraparte_region, contraparte_cuenta_id (sin FK), creado_en; FK (region, cuenta_id) → cuenta	RBR, derivada





Tres decisiones:

- **region va primero en la PK de las tres relaciones fragmentadas.** Lección del Lab 1: REGIONAL BY ROW AS `region` reparte por esa columna y, al ir primero en la clave, las filas de una región quedan juntas. Toda operación O1–O8 conoce la región, así que filtra por ella y no hay *scatter*.
- **Las FK de cuenta y movimiento son compuestas e incluyen region.** No es solo integridad referencial: **obliga** a que una cuenta viva en la región de su dueño, y un movimiento en la de su cuenta. Es la FK la que hace cumplir la fragmentación derivada de §3 dentro del motor.
- **La PII vive solo en cliente;** `cuenta` y `movimiento` referencian `cliente_id`, nunca copian nombre ni documento. La residencia se reduce a controlar una relación. `documento` es UNIQUE global: el alta (O8, 5 %) paga una verificación entre regiones para que lo frecuente siga siendo local.

3. Predicados de fragmentación

Condiciones escritas como en SQL, según pide el enunciado. $\times$ = semi-join.

Relación	Tipo	Fragmento	Predicado / definición
<code>cliente</code>	horizontal primaria	<code>cliente_sj</code>	<code>region = 'cr-sj'</code> — clientes que abrieron en San José
		<code>cliente_limon</code>	<code>region = 'cr-limon'</code> — ... en Limón
		<code>cliente_us</code>	<code>region = 'us-east'</code> — ... en Estados Unidos
<code>cuenta</code>	horizontal derivada	<code>cuenta_r</code>	<code>\$cuenta \times_{(region,,cliente_id)} cliente_r\$</code> — cuentas cuyo dueño está en <code>r</code>
<code>movimiento</code>	horizontal derivada	<code>movimiento_r</code>	<code>\$movimiento \times_{(region,,cuenta_id)} cuenta_r\$</code> — movimientos de cuentas de <code>r</code>
<code>moneda</code>	ninguna	<code>moneda</code>	replicación completa en las tres regiones (GLOBAL)

Por qué region. Es el atributo del predicado de todas las operaciones frecuentes, fija la regla de residencia y es estable. Sus tres valores son los minterms del atributo: el tipo `crdb_internal_region` no admite otro valor y la

columna es NOT NULL. Sobre eso se apoya la tabla de verificación (completitud, reconstrucción y disyunción) de cliente. Descartados: moneda (partiría las operaciones de un mismo cliente) y rango de documento (no se relaciona con dónde se originan las operaciones).

Derivada = selección directa. CockroachDB no ejecuta semi-joins para colocar filas; necesita la columna region. Gracias a la FK compuesta, cuenta.region es siempre igual a la del dueño, por lo que

```
cuenta_r = cuenta × cliente_r = σ region = 'r' (cuenta) (igual para movimiento)
```

La igualdad se sostiene **porque existe la FK compuesta**. Por transitividad, toda la historia de un cliente queda en su región hogar. En la tabla de verificación esto se comprueba con una consulta: cuentas cuya region no coincide con la de su cliente → debe dar cero.

Doble asiento en transferencias. Una transferencia de A (cr-sj) a B (cr-limon) genera **dos** movimientos en la misma transacción: TRANSF_ENVIO en la región de A y TRANSF_RECIBO en la de B, cada uno con contraparte_* apuntando al otro. Si un movimiento pudiera pertenecer a dos cuentas de regiones distintas estaría en dos fragmentos y se rompería la disyunción. Por eso contraparte_cuenta_id no lleva FK. Para E3: O5 escribe en dos fragmentos de dos regiones (la "escritura que cruza región"); O4 escribe las mismas cuatro filas en un solo fragmento (la "escritura local").

4. Decisiones sobre la PII y la residencia

Fragmentación vertical de la PII: no se aplica. cliente se mantiene como una sola relación. La PII ya está confinada a ella por diseño (§2): cuenta y movimiento solo referencian cliente_id. Lo que la regla de residencia exige —que nombre y documento vivan en la región de apertura— lo cumple la fragmentación horizontal de cliente; separar cliente_identidad de cliente_base no agregaría residencia, solo columnas repartidas dentro de la misma región. Además, las dos operaciones que leen PII (O6, identificar por documento; O8, alta) leen la fila completa, así que la vertical añadiría un join en cada una sin ahorrar ninguna lectura.

PLACEMENT RESTRICTED: evaluado y no aplicado. Esta sentencia ordena colocar todas las réplicas de cada partición REGIONAL BY ROW dentro de su región hogar. Con un nodo por región no hay dónde poner tres votantes en una sola región: la restricción no puede satisfacerse y el motor deja los rangos como están. La evidencia de E2 lo muestra: la configuración de zona declara num_voters = 3 con voter_constraints en la región hogar (e2-inspect.txt §5), pero SHOW RANGES observa tres votantes repartidos en cr-sj, cr-limon y us-east (§6). Aplicar la sentencia cambiaría lo declarado sin cambiar lo observado.

Límite que se declara. Los datos de identificación del cliente tienen región hogar en su región de apertura y su leaseholder se coloca allí. En este clúster de un nodo por región **no** puede afirmarse que las copias físicas de la PII no salgan de la región: hoy existen réplicas en las tres. Ninguna de las dos decisiones altera la evidencia de E2/E3 ni bloquea E4.

5. Tabla de verificación

Aplica a las tres relaciones fragmentadas (cliente primaria; cuenta y movimiento derivadas). moneda no se fragmenta. Consultas: p1/sql/verificacion.sql. Salida sobre los datos cargados: evidence/p1/e1-verificacion.txt (la genera p1/evidence.sh).

Propiedad	¿Se cumple?	Justificación
Compleitud	sí	Los predicados <code>region = 'cr-sj'</code> , <code>= 'cr-limon'</code> y <code>= 'us-east'</code> cubren todo el dominio de <code>region</code> : la columna es NOT NULL y su tipo (<code>crdb_internal_region</code>) solo admite esas tres regiones. Ninguna fila puede quedar fuera de un fragmento. En las derivadas, toda cuenta tiene un dueño (FK) y toda cuenta pertenece al fragmento de su dueño, así que tampoco sobra ninguna. Prueba: 0 filas fuera de fragmento en las tres tablas.
Reconstrucción	sí	Horizontal: la relación es la unión de sus fragmentos, $R = R_{\{sj\}} \cup R_{\{limon\}} \cup R_{\{us\}}$. No hace falta ningún join, porque cada fragmento conserva todas las columnas y su PK. Prueba: la suma de fragmentos iguala el total (3 000 / 4 527 / 91 008) y la diferencia tabla EXCEPT unión da 0 filas.
Disyunción	sí	Los predicados son igualdades sobre un mismo atributo con valores distintos: una fila no puede cumplir dos a la vez. En las derivadas, cada hija tiene un único padre (FK compuesta), así que cae en un solo fragmento. El doble asiento (§3) mantiene esto en las transferencias entre regiones: son dos movimientos, uno por fragmento. Prueba: 0 identificadores con más de una región.

Consultas de prueba (extracto; el archivo tiene las tres tablas):

```
-- Compleitud: filas que no caen en ningún fragmento → 0
SELECT count(*) FROM cliente
WHERE region IS NULL OR region NOT IN ('cr-sj', 'cr-limon', 'us-east');

-- Reconstrucción: filas de la tabla que no salen de la unión de fragmentos → 0
SELECT count(*) FROM (
  SELECT region, cliente_id FROM cliente
  EXCEPT
  SELECT region, cliente_id FROM (
    SELECT region, cliente_id FROM cliente WHERE region = 'cr-sj'
    UNION ALL SELECT region, cliente_id FROM cliente WHERE region = 'cr-limon'
    UNION ALL SELECT region, cliente_id FROM cliente WHERE region = 'us-east') u);

-- Disyunción: misma entidad en más de un fragmento → 0
SELECT count(*) FROM (
  SELECT cliente_id FROM cliente GROUP BY cliente_id HAVING count(DISTINCT region) >
1);
```

```
-- Derivada: cuentas fuera del fragmento de su dueño → 0 (igual para movimiento/cuenta)
SELECT count(*) FROM cuenta c JOIN cliente k ON k.cliente_id = c.cliente_id
WHERE k.region <> c.region;
```

Resultado (e1-verificacion.txt, 2026-09-18):

Comprobación	cliente	cuenta	movimiento
Filas fuera de todo fragmento	0	0	0
Total / suma de fragmentos	3 000 / 3 000	4 527 / 4 527	91 008 / 91 008
Filas que no salen de la unión	0	0	0
Identificadores en más de un fragmento	0	0	0
Filas fuera del fragmento de su padre	—	0	0

movimiento tiene 91 008 filas y no las 90 000 del seed: cada transferencia medida en E3 agrega dos movimientos. La verificación se hace sobre el estado real, no sobre el recién cargado. La misma consulta cuenta las transferencias entre regiones por par (cr-sj → cr-limon: 2 524, etc.). Cada una dejó su envío y su recibo en fragmentos distintos, y ninguna fila aparece en dos.

6. Réplica

Qué se replica, con qué factor y por qué. "Declarado" es lo que pide la configuración de zona (e2-inspect.txt §5); "observado" es lo que muestra SHOW RANGES (§6).

Tabla / fragmento	Localidad	Réplicas declaradas	Réplicas observadas	Lectura servida desde	Criterio de costo
moneda (completa)	GLOBAL	una por región (global_reads = true)	3 votantes, una por región	cualquier región, localmente	Replicar completo: se lee desde todas las regiones y casi no se escribe
cliente_r	RBR	3 votantes en r, 5 réplicas en total	3 votantes, uno por región	leaseholder en r	Un fragmento por región; se replica por disponibilidad, no para leerlo lejos
cuenta_r,	RBR	igual que	igual	leaseholder en	Igual; viajan con su cliente

Tabla / fragmento	Localidad	Réplicas declaradas	Réplicas observadas	Lectura servida desde	Criterio de costo
movimiento_r	(derivada)	cliente_r		r	
p1_control.folio_comprobante	sin regiones	num_replicas = 3	3 votantes	leaseholder (uno)	Factor 3 exacto para medir la mayoría 2/3 en E4

moneda: replicación completa en las tres regiones. Es la única tabla que se replica para *leer cerca*. Los depósitos, retiros y transferencias (O3–O5) consultan la tasa (O7, ~300 % según §1) en las tres regiones. Se escribe una vez al día. Si tuviera una sola región hogar, cada transferencia desde `cr-limon` o `us-east` haría un join remoto contra `cr-sj` para convertir el monto. Con `GLOBAL`, cada nodo sirve la lectura con su propia copia, sin salir de la región. El precio lo pagan las escrituras: un `UPDATE` de moneda espera a que la escritura sea visible en todas las réplicas (*commit wait*) y es más lento que una escritura RBR. Con ~1 escritura diaria frente a cientos de lecturas por cada mil operaciones, el costo se paga en la operación más rara. La tabla es pequeña (3 filas), así que tenerla completa en cada región casi no cuesta disco.

cliente, cuenta, movimiento: fragmento en su región, sin copias para lectura remota. No se replican para que otra región los lea: el ~95 % de las operaciones es local (§1), y replicarlos fuera de su región iría **contra** la regla de residencia de la PII. Cada fragmento tiene su leaseholder en su región hogar, que es la copia que atiende lecturas y coordina escrituras. Las réplicas adicionales (factor 3) existen solo por **disponibilidad**: si cae el nodo del leaseholder, otra réplica con los datos confirmados toma el lease. Esto es lo que mide E4. La consecuencia medida en E3: leer un fragmento de otra región cuesta el salto hasta su leaseholder (p99 ~3× el local). Se acepta porque esa lectura es rara (O9, reporte batch).

Factor 3 y no más. Con 3 réplicas, una escritura se confirma cuando la tienen 2 (mayoría). Así se tolera la caída de 1 nodo sin perder escrituras confirmadas. Factor 5 toleraría 2 caídas, pero cada escritura esperaría a 3 réplicas y harían falta al menos 5 nodos. Con 3 nodos, 3 es el máximo útil.

Declarado ≠ observado en RBR. La configuración pide 3 votantes **dentro** de la región hogar y 5 réplicas en total. Con un nodo por región eso no se puede cumplir, y el motor deja 3 votantes repartidos en las tres regiones (el límite de §4). La tabla de E4 se sacó a una base sin regiones (`p1_control`) precisamente para medir la falla sobre un factor 3 que sí se cumple como se declara.

7. Réplica de Özsü frente a réplica Raft

La palabra "réplica" aparece en §6 con dos sentidos. Mezclarlos lleva a respuestas contradictorias ("¿`cliente_limon` está replicado?": sí y no). Se separan así:

	Réplica de Özsü (diseño distribuido)	Réplica Raft (CockroachDB)
Unidad	El fragmento lógico	El rango : un tramo contiguo de claves (hasta 512

	Réplica de Özsu (diseño distribuido)	Réplica Raft (CockroachDB)
	(cliente_limon, moneda)	MiB)
Quién decide	El diseñador, en la fase de asignación	El <i>allocator</i> del motor, dentro de la configuración de zona
Para qué	Acceso: leer cerca y evitar joins remotos; también disponibilidad	Durabilidad y disponibilidad: sobrevivir a la caída de nodos
Quién atiende lecturas	Cualquier copia (la del sitio que consulta)	Solo el leaseholder ; las demás no sirven lecturas consistentes (salvo <code>GLOBAL</code> o <i>follower reads</i>)
Propagación de escrituras	Se elige: <i>eager</i> o <i>lazy</i> ; ROWA, copia primaria o votación	Fija: <i>eager</i> con líder único, confirmada por mayoría (2 de 3)
Criterio	Costo: razón lecturas/escrituras × costo de comunicación	Tolerancia: $RF = 2f + 1$ réplicas para tolerar f caídas

En la clasificación de Özsu, Raft es un protocolo **eager de copia primaria** (el líder/leaseholder ordena todas las escrituras). Se distingue de ROWA en que confirma con la mayoría y no espera a todas las copias. Por eso tolera la caída de una réplica sin bloquear (E4).

Cómo se aplica a cada tabla del diseño:

Tabla / fragmento	Réplica de Özsu (asignación)	Réplica Raft (observada)	Lectura
moneda	Replicación completa: una copia por región	3 votantes, uno por región, <code>global_reads = true</code>	Cada nodo la sirve con su copia
cliente_r, cuenta_r, movimiento_r	Sin replicación: un solo sitio, la región <code>r</code>	3 votantes (uno por región); leaseholder en <code>r</code>	Solo el leaseholder en <code>r</code>

- **moneda es el único caso donde las dos coinciden.** `GLOBAL` implementa la replicación completa de Özsu con réplicas Raft que sí atienden lecturas locales (lecturas no bloqueantes). Por eso se pagan escrituras lentas (§6).
- **Los fragmentos RBR no están replicados en el sentido de Özsu.** Cada uno está asignado a un sitio, que es el único que lo atiende. Las copias Raft en las otras dos regiones no son "réplicas para leer lejos": solo votan para que el rango tenga mayoría. La lectura remota de E3 lo prueba: desde `cr-sj`, leer una cuenta de `cr-limon` va hasta el leaseholder en `crdb-2` aunque `crdb-1` tenga una copia Raft de ese mismo rango.
- **La residencia se evalúa sobre la réplica Raft, no sobre la de Özsu.** Lógicamente, la PII de `cliente_limon` vive en un solo sitio. Físicamente, sus bytes están en los tres nodos. Es el límite

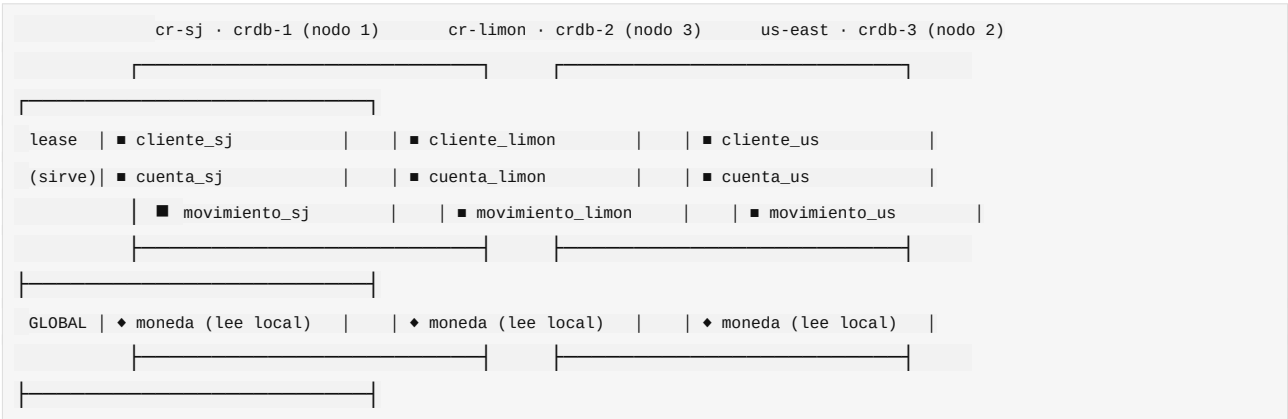
declarado en §4 y la razón por la que E5 pide 3 nodos por región con `PLACEMENT RESTRICTED` para cumplirla.

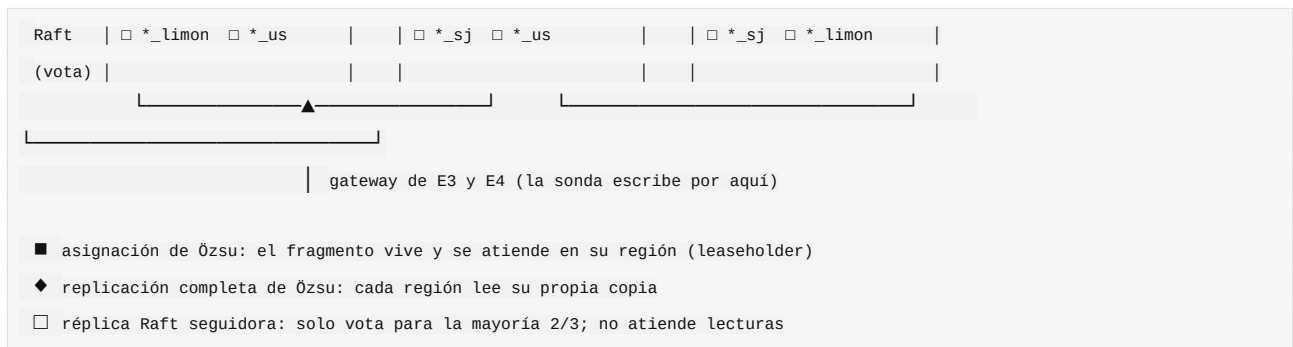
8. Asignación: fragmento → región → nodo

La asignación **declarada** es la configuración de zona de cada partición (`lease_preferences = [[+region=r]]`, `e2-inspect.txt §5`). La **observada** sale de `p1/sql/asignacion.sql`: codifica cada fila a su clave física (`crdb_internal.encode_key`), busca el rango que la contiene y cruza ese rango con `SHOW RANGES ... WITH DETAILS`. No se deduce del `start_key`, porque el motor fusiona rangos vacíos y un rango puede empezar antes del prefijo de su partición. Evidencia: `evidence/p1/e1-asignacion.txt` (la genera `p1/evidence.sh`).

Fragmento	Región hogar	Nodo (contenedor)	Rango	Filas Leaseholder	Votantes
cliente_sj, cuenta_sj, movimiento_sj	cr-sj	nodo 1 (crdb-1)	124, 132, 136	1 473 / 2 205 / 43 996 nodo 1 · cr-sj	{1, 2, 3}
cliente_limon, cuenta_limon, movimiento_limon	cr-limon	nodo 3 (crdb-2)	123, 131, 135	922 / 1 421 / 29 896 nodo 3 · cr-limon	{1, 2, 3}
cliente_us, cuenta_us, movimiento_us	us-east	nodo 2 (crdb-3)	125, 133, 137	605 / 901 / 17 116 nodo 2 · us-east	{1, 2, 3}
moneda (completa)	todas	los 3	121	3 nodo 1 (coordina escrituras)	{1, 2, 3}

La consulta de resumen (`asignacion.sql §3`) cuenta las filas RBR cuyo leaseholder está fuera de su región hogar: **0**. Cada fragmento ocupa exactamente un rango y lo atiende el nodo de su región. Los `node_id` son los de esta máquina: el clúster los asigna al unirse cada nodo, y en otra máquina `crdb-2` fue el nodo 2 (E3 §4). La asignación es **por región**; el número de nodo es incidental.





Con la topología que RBR espera (≥ 3 nodos por región y `PLACEMENT RESTRICTED`), las flechas punteadas quedarían **dentro** de cada caja: los tres votantes de `cliente_limon` estarían en `cr-limon`. Así se cumpliría la residencia, pero una caída de toda la región dejaría sus filas sin servicio. E4 §"Caída de una región" mide el caso contrario, el de este montaje.

Anexo — Borrador de DDL (No es el `schema.sql` final, pero es basado en la propuesta presentada)

```
CREATE TABLE moneda (
  codigo      CHAR(3)      PRIMARY KEY,
  nombre      STRING      NOT NULL,
  tasa_crc    DECIMAL(12,4) NOT NULL CHECK (tasa_crc > 0),
  actualizado_en TIMESTAMPTZ NOT NULL DEFAULT now()
) LOCALITY GLOBAL;

CREATE TABLE cliente (
  region      crdb_internal_region NOT NULL,
  cliente_id  UUID                NOT NULL DEFAULT gen_random_uuid(),
  documento   STRING             NOT NULL,
  nombre      STRING             NOT NULL,
  email       STRING             NOT NULL,
  creado_en   TIMESTAMPTZ        NOT NULL DEFAULT now(),
  PRIMARY KEY (region, cliente_id),
  UNIQUE (documento)
) LOCALITY REGIONAL BY ROW AS region;

CREATE TABLE cuenta (
  region      crdb_internal_region NOT NULL,
  cuenta_id   UUID                NOT NULL DEFAULT gen_random_uuid(),
  cliente_id  UUID                NOT NULL,
  moneda      CHAR(3)             NOT NULL REFERENCES moneda (codigo),
  saldo       DECIMAL(18,2)        NOT NULL CHECK (saldo >= 0),
  estado      STRING              NOT NULL DEFAULT 'ACTIVA'
  CHECK (estado IN ('ACTIVA', 'BLOQUEADA', 'CERRADA')),
  abierta_en  TIMESTAMPTZ         NOT NULL DEFAULT now(),
  PRIMARY KEY (region, cuenta_id),
  FOREIGN KEY (region, cliente_id) REFERENCES cliente (region, cliente_id)
) LOCALITY REGIONAL BY ROW AS region;

CREATE TABLE movimiento (
```

```
region          crdb_internal_region NOT NULL,
movimiento_id   UUID                  NOT NULL DEFAULT gen_random_uuid(),
cuenta_id       UUID                  NOT NULL,
tipo            STRING                 NOT NULL
CHECK (tipo IN
('DEPOSITO', 'RETIRO', 'TRANSF_ENVIO', 'TRANSF_RECIBO')),
monto           DECIMAL(18,2) NOT NULL CHECK (monto > 0),
moneda          CHAR(3)              NOT NULL REFERENCES moneda (codigo),
saldo_resultante DECIMAL(18,2) NOT NULL CHECK (saldo_resultante >= 0),
contraparte_region crdb_internal_region,
contraparte_cuenta_id UUID,
creado_en       TIMESTAMPTZ          NOT NULL DEFAULT now(),
PRIMARY KEY (region, movimiento_id),
FOREIGN KEY (region, cuenta_id) REFERENCES cuenta (region, cuenta_id),
INDEX movimiento_por_cuenta (region, cuenta_id, creado_en DESC) -- sirve a 02
) LOCALITY REGIONAL BY ROW AS region;
```

E2 — Implementación y reproducción (20 %)

Grupo 3 · TI-4601 Bases de Datos Avanzadas. Dominio: **Opción A**. Regiones: cr-sj, cr-limon, us-east.

Diseño de fragmentación: <docs/E1-diseno.md>.

Esta guía permite levantar el sistema, cargar los datos y repetir las mediciones en una máquina limpia. Todos los comandos se ejecutan en el **host**, desde la **raíz del repositorio**.

Requisitos

- Docker Engine + Docker Compose v2 (`docker compose version`).
- Puertos libres 26257 y 8080 (y 5458 para PostgreSQL en E5).
- Recomendado: 4 CPU, 4 GiB de RAM libres y 5 GiB de disco.
- Archivo `.env` en la raíz con `COCKROACH_LICENSE=<licencia del docente>` (ver `.env.example`). Sin licencia, `REGIONAL BY ROW` / `GLOBAL` solo funcionan los primeros 7 días del clúster. **No versionar `.env`**.

No hace falta instalar Python ni psycopg en el host: todo corre dentro del contenedor `app-crdb`.

1. Configurar todo (E2)

```
p1/setup.sh
```

Paso	Qué hace	Archivo
1	Levanta los 3 nodos del clúster del curso (<code>make lab1-up</code>)	<code>docker-compose.yml</code>
2	Espera a que los 3 nodos estén vivos	—
3	Instala la licencia sin imprimirla	<code>labs/lab1-cluster/install_license.py</code>
4	Crea <code>p1_banca</code> con región primaria <code>cr-sj</code> y las otras dos regiones	sql/00_database.sql
5	Crea <code>moneda</code> (<code>GLOBAL</code>); <code>cliente</code> , <code>cuenta</code> y <code>movimiento</code> (<code>REGIONAL BY ROW</code>); y la tabla de E4	sql/01_schema.sql , sql/02_e4_table.sql
6	Genera y carga los datos con semilla fija	gen/seed.py
7	Verifica clúster, esquema, fragmentación y datos	check.py

Tiempo aproximado: menos de 1 minuto con las imágenes ya descargadas.

Resultado esperado:

```
Generado en ... s: 3000 clientes, 4527 cuentas, 90000 movimientos
Huella de datos (semilla=4601, clientes=3000): ad995e29a16440f7
...
Resultado: 10/10 verificaciones.
```

La **huella** debe ser exactamente `ad995e29a16440f7`: indica que se generaron los mismos datos que en nuestras mediciones. El script es idempotente: si se ejecuta otra vez, no duplica nada. Para recargar los datos desde cero: `p1/setup.sh --reset`.

El proyecto usa sus propias bases (`p1_banca` y `p1_control`) y no modifica `ti4601`, que usan los labs.

2. Evidencia de la implementación

```
p1/evidence.sh
```

Genera, con marca de tiempo:

Archivo	Contenido
<code>evidence/p1/e2-node-status.txt</code>	Contenedores y <code>cockroach node status</code>
<code>evidence/p1/e2-inspect.txt</code>	<code>SHOW REGIONS</code> , localidad de cada tabla, <code>SHOW CREATE TABLE</code> , filas por fragmento, configuración de zona y <code>SHOW RANGES ... WITH DETAILS</code>
<code>evidence/p1/e2-check.txt</code>	Salida del verificador (10 comprobaciones)
<code>evidence/p1/e1-verificacion.txt</code>	Compleitud, reconstrucción, disyunción y derivada de la tabla de verificación del E1 (<code>sql/verificacion.sql</code>)
<code>evidence/p1/e1-asignacion.txt</code>	Asignación observada del E1 §8: cada fragmento → rango → leaseholder y votantes (<code>sql/asignacion.sql</code>)

3. Mediciones de latencia (E3)

Espere a que el clúster esté en reposo: unos minutos después de `setup.sh`, o hasta que `docker stats` muestre poca CPU en los nodos. Justo después de cargar datos o de borrar una base, el motor hace trabajo de fondo que infla las escrituras.

```
docker compose --profile lab1 run --rm --no-deps app-crdp python3 p1/bench/latency.py
```

Genera `evidence/p1/e3-latency.csv` (400 muestras crudas) y `evidence/p1/e3-latency-summary.txt` (entorno, método y tabla p50/p99). Opciones: `--corridas` (por defecto 100, mínimo 30), `--warmup` (10), `--`

gateway (crdb-1) y `--prefijo` (para no sobrescribir una corrida anterior).

Definición: una operación es *local* si todas las filas que toca tienen como región hogar la región del gateway SQL; *cruza región* si alguna fila tiene otra región hogar.

Caso	Operación del E1	Filas
lectura-local	O1: saldo de una cuenta <code>cr-sj</code> desde <code>crdb-1</code>	1 lectura
lectura-remota	O1: saldo de una cuenta <code>cr-limon</code> desde <code>crdb-1</code>	1 lectura
escritura-local	O4: transferencia <code>cr-sj</code> ↔ <code>cr-sj</code>	2 UPDATE + 2 INSERT
escritura-cruza	O5: transferencia <code>cr-sj</code> ↔ <code>cr-limon</code>	2 UPDATE + 2 INSERT

Método:

- Gateway fijo (`crdb-1`) y una conexión persistente.
- Warm-up descartado.
- Orden de los casos aleatorio en cada ronda.
- `perf_counter_ns` alrededor de la operación completa, incluidos el COMMIT y los reintentos.
- p99 por *nearest rank*.
- Antes de medir, se comprueba que el leaseholder de cada cuenta usada esté en su región hogar.
- Las transferencias alternan de dirección para que los saldos no se agoten.

4. Falla de un nodo (E4)

```
p1/chaos/falla.sh          # corrida 1: cae el nodo de cr-limon durante 15 s
p1/chaos/falla.sh --corrida 2  # repeticiones, sin pisar la evidencia anterior
```

Paso Qué hace

- 1 Comprueba que haya 3 nodos vivos
- 2 Busca por **localidad** el `node_id`, el `store_id` y el contenedor del nodo que caerá (el número del contenedor no es el del nodo)
- 3 Hace una escritura sana y mueve el lease de `folio_comprobante` a ese nodo (`ALTER RANGE RELOCATE LEASE`)
- 4 Lanza la sonda: toma folios cada 0,2 s por `crdb-1`, con `statement_timeout` de 2 s
- 5 `docker stop --timeout 0` y guarda la época del stop; `CAIDO` segundos después, `docker start`

Paso Qué hace

6 Guarda el estado final: nodos, lease y último folio confirmado

7 Calcula el RTO desde el CSV y muestra la resta (`chaos/rto.py`)

Genera `evidence/p1/e4-corrida<N>-antes.txt`, `.csv`, `-sonda.txt`, `-stop.epoch`, `-stop.txt`, `-despues.txt` y `-rto.txt`. Si el script se interrumpe, vuelve a arrancar el nodo al salir. Opciones: `--region` (por defecto `cr-limon`), `--caido` (15 s) y `--corrida`.

El RPO se calcula aparte, sobre la evidencia de la corrida:

```
docker compose --profile lab1 run --rm --no-deps app-crdb python3 p1/chaos/rpo.py \
  evidence/p1/e4-corrida1.csv evidence/p1/e4-corrida1-stop.epoch evidence/p1/e4-corrida1-despues.txt \
  | tee evidence/p1/e4-corrida1-rpo.txt
```

Caída de una región completa (opcional):

```
p1/chaos/region.sh # cae cr-limon durante 30 s
```

A diferencia de `falla.sh`, no mueve ningún lease: observa la colocación diseñada (E1 §8) y mide seis casos en paralelo (lectura y escritura de la región viva, de la caída, la que cruza y moneda GLOBAL), cada uno en su hilo y su conexión. Al final espera a que los leases vuelvan a su región y corre `check.py`. Genera `evidence/p1/e4-region-<region>-*`. Resultados en [docs/E4-falla.md](#).

5. Comparación con PostgreSQL de un nodo (E5)

Requiere el clúster arriba y con datos (sección 2), porque la foto de costo mide los dos motores a la vez.

```
make up
docker compose run --rm app python3 p1/baseline/seed_pg.py # huella esperada: ad995e29a16440f7
docker compose run --rm app python3 p1/baseline/latency_pg.py
p1/baseline/docker_stats.sh
```

Corre contra PostgreSQL **el mismo esquema, los mismos datos y las mismas cuatro operaciones** de E2/E3, para que la diferencia medida venga del motor y no de otros datos u otras operaciones.

Paso Qué hace

Archivo

1 Levanta PostgreSQL 16

`docker-compose.yml`

2 Crea `p1_banca` con el esquema traducido (sin LOCALITY) y carga los datos con las funciones de `gen/seed.py`

[baseline/seed_pg.py](#),

Paso	Qué hace	Archivo
		baseline/01_schema_pg.sql
3	Mide con las operaciones de <code>bench/latency.py</code> , en <code>SERIALIZABLE</code> como CockroachDB	baseline/latency_pg.py
4	<code>docker stats</code> en reposo y bajo carga, volúmenes y piezas a operar	baseline/docker_stats.sh

Genera `evidence/p1/e5-latency-postgres.csv`, `evidence/p1/e5-latency-postgres-summary.txt` y `evidence/p1/e5-docker-stats.txt`. La comparación está en [docs/E5-comparacion.md](#).

Cómo leer los casos en PostgreSQL: todas las filas están en el mismo servidor. Los cuatro casos se conservan para comparar operación por operación con E3, pero *local* y *cruza* solo describen la región hogar de las filas, no una distancia.

6. Límites que hay que declarar

- **Las regiones son lógicas.** Los 3 nodos corren en la misma máquina y no hay latencia inyectada, así que la diferencia local/remoto medida no representa una WAN.
- **Las particiones REGIONAL BY ROW no cumplen su configuración de zona.** Cada una pide 5 réplicas y 3 votantes dentro de su región hogar (`SHOW ZONE CONFIGURATIONS`), pero hay un solo nodo por región. Lo observado en `SHOW RANGES` es: 3 votantes, uno por nodo, con el leaseholder en la región hogar. Por eso las escrituras locales también esperan la confirmación de nodos de otras regiones.
- **No hay garantía de residencia.** Por el punto anterior, hay copias de la PII en las tres regiones. Este montaje demuestra la región hogar, no el cumplimiento de la regla de residencia.
- **La prueba de falla (E4) usa `p1_control.folio_comprobante`,** que está en una base sin regiones con `num_replicas = 3` y sí cumple su configuración (mayoría 2/3).
- **E5 compara contra un PostgreSQL con configuración por defecto,** sin los límites de memoria de los nodos (`--cache=256MiB`). La réplica de lectura de la alternativa no se montó: con replicación asíncrona no cambia la latencia de escritura del primario.
- **El volumen `ti4601_pgdata` también contiene la base `ti4601` de los labs,** así que su tamaño es una cota superior del costo en disco de `p1_banca`.

Estructura

```
p1/
├─ README.md          esta guía
├─ setup.sh           configuración completa (E2)
├─ evidence.sh        evidencia de E1 (verificación y asignación) y E2
├─ check.py           verificador de solo lectura
├─ docs/              E1-diseno, E3-mediciones, E4-falla, E5-comparacion
├─ chaos/             falla de un nodo (falla, sonda, rto, rpo) y caída de región
                       (region) – E4
```

— baseline/	PostgreSQL de un nodo (E5)
— sql/	00_database, 01_schema, 02_e4_table, inspect, verificacion, asignacion
— gen/seed.py	generador determinista
— bench/latency.py	mediciones de E3
— informe.sh	arma entregas/p1/INFORME.md y .pdf
— informe/	portada, apéndice y render a HTML del informe
evidence/p1/	salidas generadas por los scripts

Problemas comunes

Síntoma	Qué hacer
ERROR: no hay 3 nodos vivos	make lab1-status; esperar 20 s y repetir p1/setup.sh
Error de licencia en multi-región	Revisar .env y repetir p1/setup.sh
La huella no coincide	Verificar que no se pasaron --semilla o --clientes distintos
[FAIL] folio_comprobante ... votantes	Esperar 1–2 minutos (reubicación de réplicas) y ejecutar check.py otra vez
Un nodo quedó detenido	docker start ti4601-crdb-1 ti4601-crdb-2 ti4601-crdb-3
Escrituras con p99 de cientos de ms	Clúster aún ocupado tras la carga; esperar y repetir con otro --prefijo

E3 — Mediciones de latencia (20 %)

Datos: `evidence/p1/e3-latency-summary.txt` (corrida principal) y `evidence/p1/otra-maquina/e3-latency-summary.txt` (reproducción en una segunda máquina). Script: `p1/bench/latency.py`. Método completo en `p1/README.md §3`.

Definición. Una operación es *local* si todas las filas que toca tienen como región hogar la región del gateway SQL (`crdb-1`, `cr-sj`); *cruza región* si alguna fila tiene otra región hogar (`cr-limon`).

Tabla de resultados (formato del enunciado)

Corrida principal — 2026-09-16, 4 CPU / 11,8 GiB, n = 100 por caso, warm-up 10 descartado:

Desde región	Operación del dominio	p50 (ms)	p99 (ms)	n	Notas
Lectura local · R1	O1 saldo de cuenta <code>cr-sj</code>	1.418	6.378	100	0 reintentos
Lectura remota · R1 → R2	O1 saldo de cuenta <code>cr-limon</code>	1.971	18.897	100	17/100 por encima de 10 ms
Escritura local · R1	O4 transferencia <code>cr-sj ↔ cr-sj</code>	18.510	38.627	100	2 UPDATE + 2 INSERT, una transacción
Escritura que cruza región · R1 → R2	O5 transferencia <code>cr-sj ↔ cr-limon</code>	19.071	37.310	100	mismo trabajo que O4

Reproducción — 2026-09-17, otra máquina (12 CPU / 7,9 GiB), mismo script, misma semilla, misma huella de datos:

Desde región	p50 (ms)	p99 (ms)	n	Notas
Lectura local · R1	0.770	6.458	100	
Lectura remota · R1 → R2	1.520	17.342	100	17/100 por encima de 10 ms
Escritura local · R1	16.467	31.236	100	
Escritura que cruza región · R1 → R2	15.073	37.296	100	

Razones remota/local (principal · reproducción): lectura p50 **1.4 · 2.0**, p99 **3.0 · 2.7**; escritura p50 **1.03 · 0.92**, p99

Interpretación

1. En lectura, el p99 remoto sí es mayor, y la diferencia está en la cola, no en la mediana. La mediana remota es apenas 1,4–2× la local, pero el p99 es ~3× y el patrón es idéntico en las dos máquinas: 17 de 100 lecturas remotas tardan 11–19 ms en lugar de ~1,5 ms. Una lectura de una fila con hogar en `cr-limon` entra por el gateway de `cr-sj` y debe ser atendida por el leaseholder de ese rango, que está en `crdb-2` (comprobado antes de medir: "leaseholder cuenta cr-limon: en región hogar"). Ese salto adicional gateway → leaseholder es lo que la lectura local no paga; en una máquina compartida el costo aparece cuando el salto compite con el trabajo de fondo de los otros nodos, es decir, en el p99. Es la razón por la que el enunciado pide mediana y p99: el promedio (4,9 ms) no muestra ni lo típico ni la cola.

2. En escritura, local ≈ cruza región. No es un error de medición: es la configuración de zona. Cada partición `REGIONAL BY ROW` declara `num_voters = 3` con `voter_constraints` en su región hogar (`evidence/p1/e2-inspect.txt §5`), pero hay un solo nodo por región. Lo observado en `SHOW RANGES (§6)` son 3 votantes, uno por nodo, en tres regiones distintas. Un commit necesita mayoría 2 de 3, así que la transferencia "local" `cr-sj ↔ cr-sj` también espera la confirmación de un nodo de otra región — el mismo viaje que la transferencia que cruza. Con esta topología, "escritura local" es local en el hogar de las filas pero no en el quórum. Con varios nodos por región (lo que RBR espera), los votantes cabrían en la región hogar y la diferencia aparecería.

3. La diferencia dominante es lectura frente a escritura: 13–21× en la mediana. Es el costo de replicar la entrada y esperar que una mayoría la persista (consenso + durabilidad), igual que en el Lab 1. Ninguna transferencia necesitó reintento por `SerializationFailure` (columna `reintentos = 0` en las 400 muestras de cada corrida).

4. La reproducción confirma el método. Otra máquina, con el triple de CPU y menos memoria, dio las mismas razones y el mismo 17/100 en la cola remota. Las medianas absolutas bajan (0,77 vs 1,42 ms en lectura local) porque cambia el hardware; las relaciones entre casos no. Un detalle operativo de esa máquina: allí `node 2 = cr-limon` y `node 3 = us-east`, al revés que en la corrida principal; el `node_id` lo asigna el clúster, no Compose, y E4 debe buscar nodo y `store_id` por localidad.

Límites que se declaran

- Las regiones son lógicas: tres contenedores en una computadora, sin latencia inyectada. Ninguna de estas cifras representa una WAN; la razón remota/local en escritura ≈ 1 sería distinta con distancia real.
- Misma máquina y mismos límites de recursos dentro de cada corrida (`--cache=256MiB`, `--max-sql-memory=256MiB`); perfil: clúster completo del Lab 1.
- Warm-up de 10 rondas descartado; orden de casos aleatorio por ronda; una conexión persistente al gateway (no se mide el tiempo de conexión); reloj `perf_counter_ns` alrededor de la operación completa, COMMIT y reintentos incluidos; p99 por *nearest rank*.
- Con $n = 100$, el p99 por *nearest rank* es la 99.^a muestra ordenada (una sola observación por encima). Por eso el resumen reporta también p90 y máximo, y los CSV conservan las 400 muestras.

E4 — Falla de nodo y de región (15 %)

Script: `p1/chaos/falla.sh` (orquestra), `p1/chaos/sonda.py` (escribe), `p1/chaos/rto.py` (RTO), `p1/chaos/rpo.py` (RPO) y `p1/chaos/region.sh` / `region.py` (caída de una región, opcional). Evidencia: `evidence/p1/e4-corrida{1,2,3}-*` y `evidence/p1/e4-region-cr-limon-*`. Cómo repetirla: `p1/README.md §4`.

Qué se escribe y sobre qué tabla

La sonda toma el siguiente folio de la serie `TRANSFERENCIAS` en `p1_control.folio_comprobante`:

```
UPDATE folio_comprobante SET ultimo_folio = ultimo_folio + 1, actualizado_en = now()
WHERE serie = 'TRANSFERENCIAS' RETURNING ultimo_folio;
```

Es la escritura que haría cada transferencia para numerar su comprobante: es del dominio y cada confirmación devuelve un número verificable después. La tabla está en una base sin regiones con `num_replicas = 3` (`p1/sql/02_e4_table.sql`): 3 votantes, uno por nodo, así que la mayoría 2/3 que se pone a prueba es la que declara su configuración. Sobre las tablas RBR no se podría decir lo mismo (E1 §6: declarado ≠ observado).

Protocolo (`falla.sh`)

#	Paso del enunciado	Cómo se hace	Evidencia
1	Estado sano	3 nodos vivos + una escritura de prueba <code>UPDATE ... RETURNING</code> antes de tocar nada	<code>-antes.txt</code>
—	Precondición	El lease del rango se mueve al nodo que va a caer y se comprueba que llegó	<code>-antes.txt</code>
2	Detener un nodo	<code>docker stop --timeout 0</code> del contenedor, y la época justo al terminar (<code>date +%s.%N</code>)	<code>-stop.epoch</code> , <code>-stop.txt</code>
3	Cronometrar	La sonda intenta un folio cada 0,2 s por <code>crdb-1</code> ; cada intento guarda inicio, fin, estado y folio	<code>.csv</code> , <code>-sonda.txt</code>
—	Restaurar	<code>docker start</code> 15 s después; se espera a que haya 3 nodos vivos	<code>-stop.txt</code> , <code>-despues.txt</code>
4	RTO	<code>rto.py</code> cruza el CSV con la época y muestra la resta	<code>-rto.txt</code>

Por qué mover el lease al nodo que cae. Si cae un nodo que no tiene el lease, la escritura sigue confirmándose con los otros dos votantes sin interrupción: el RTO mediría casi nada. Al detener al leaseholder, el rango se queda

sin quien coordine las escrituras hasta que otro votante toma el lease. Ese intervalo es el que se mide. Antes de la corrida el lease estaba en el nodo 2 (us-east); RELOCATE LEASE lo llevó al nodo 3 (cr-limon), como muestra antes.txt.

Por qué el gateway es otro nodo. La sonda escribe siempre por crdb-1 (cr-sj), que sigue vivo. Si el gateway fuera el nodo detenido, se mediría también cuánto tarda el cliente en reconectarse a otro, que es un problema del cliente, no del clúster.

Por qué statement_timeout = 2 s. Sin límite, el primer intento después del stop quedaría bloqueado hasta la recuperación y el CSV tendría una sola fila larga en lugar de mostrar la ventana sin servicio. El precio es la resolución: el RTO se conoce con un error de hasta un intento (ver abajo).

Nodo y store_id por localidad

El número del contenedor no es el número del nodo. node_id lo asigna el clúster según el orden en que cada nodo se une, y cambia entre máquinas: en la de la corrida principal crdb-2 es el nodo 3; en la de la reproducción de E3 era el nodo 2 (E3 §4). Además, RELOCATE LEASE recibe un store_id, no un node_id. falla.sh no asume ninguno de los dos:

```
SELECT g.node_id, s.store_id, split_part(g.address, ':', 1)
FROM crdb_internal.gossip_nodes g JOIN crdb_internal.kv_store_status s ON s.node_id =
g.node_id
WHERE g.locality LIKE 'region=cr-limon,%';
-- 3 | 3 | crdb-2
```

Con la dirección que anuncia el nodo (crdb-2), busca el contenedor cuyo hostname coincide (docker inspect -f '{{.Config.Hostname}}'). Salida de la corrida 1:

```
region=cr-limon → node_id=3, store_id=3, dirección=crdb-2, contenedor=ti4601-crdb-2
```

Corrida 1 — 2026-09-18, 4 CPU / 11,8 GiB

Línea de tiempo (UTC; filas del CSV evidence/p1/e4-corrida1.csv):

Momento	Intento Inicio → fin	Estado	Folio	Latencia
Último OK antes de la falla	28 03:01:27.453 → 27.461	ok	29	7,8 ms
docker stop termina	— 03:01:27.798	—	—	—
En curso durante el stop	29 03:01:27.653 → 29.656	error (timeout)	—	2 003 ms
Primero que empieza después	30 03:01:29.657 → 31.659	error (timeout)	—	2 003 ms
Primer OK después del stop	31 03:01:31.660 → 32.418	ok	30	758 ms

Momento	Intento	Inicio → fin	Estado	Folio	Latencia
Siguiente	32	03:01:32.418 → 32.424	ok	31	5,7 ms
<code>docker start</code>	—	03:01:43.046	—	—	—

Cálculo del RTO (evidence/p1/e4-corrida1-rto.txt):

```
RTO = fin_epoch del primer OK que empezó después del stop - época del stop
    = 1789786892.417700 - 1789786887.797719
    = 4.620 s → 4 620 ms
```

Cómo se ubica en el CSV: (1) leer `e4-corrida1-stop.epoch`; (2) buscar la primera fila con `inicio_epoch` ≥ esa época y `estado` = `ok` (intento 31); (3) restar la época a su `fin_epoch`. Se exige que el intento **empiece** después del stop: el intento 29 empezó 0,145 s antes y un OK suyo no probaría que el clúster se recuperó, solo que el nodo viejo alcanzó a confirmar.

Resolución. El intento 30 estuvo bloqueado de 29,657 a 31,659 sin confirmarse, y el 31 se confirmó a las 32,418 tras esperar 758 ms. El rango volvió a tener leaseholder en algún momento entre ~31,66 y 32,42: el RTO real está entre ~**3,9 y 4,6 s**, y 4,62 s es la cota superior que da esta sonda. Del intento 32 en adelante la latencia vuelve a la normal (5–8 ms, igual que antes de la falla), aunque el nodo siguió detenido hasta las 43,046. Es decir, el clúster atendió escrituras con **2 de 3 nodos**. Después de la falla, el lease quedó en el nodo 1 (`cr-sj`) y el rango conserva sus 3 votantes (`-despues.txt`).

Errores durante la ventana. 2 intentos fallaron, los dos con `QueryCanceled: query execution canceled due to statement timeout`: la escritura esperaba al leaseholder y la sonda la canceló a los 2 s. No hubo errores de conexión, porque el gateway nunca cayó. Esos dos intentos tienen resultado desconocido para el cliente: no se cuentan como confirmados.

Repeticiones: 3 corridas

La misma prueba se repitió con `p1/chaos/falla.sh --corrida 2` y `--corrida 3`. Esas dos corridas se hicieron en la máquina de otro integrante (`iQuick-DESKTOP`). Allí `crdb-2` es el **nodo 2** y no el 3, y `falla.sh` lo encontró igual por localidad (`-antes.txt`), sin cambiar el script.

Corrida	Fecha · máquina	Nodo caído (cr-limon)	Último OK antes	Primer OK después	Errores (timeout)	RTO
1	2026-09-18 · chris-HP (4 CPU)	nodo 3 / <code>crdb-2</code>	intento 28, folio 29	intento 31, folio 30	2	4,620 s
2	2026-09-19 · iQuick-	nodo 2 / <code>crdb-2</code>	intento 28,	intento 32, folio	3	6,530 s

Corrida	Fecha · máquina	Nodo caído (cr-limon)	Último OK antes	Primer OK después	Errores (timeout)	RTO
	DESKTOP		folio 29	30		
3	2026-09-19 · iQuick-DESKTOP	nodo 2 / crdb-2	intento 28, folio 297	intento 31, folio 298	2	5,156 s

Restas (de `-rto.txt`):

```
corrida 1: 1789786892.417700 - 1789786887.797719 = 4.620 s
corrida 2: 1789882573.314526 - 1789882566.784132 = 6.530 s
corrida 3: 1789882696.788324 - 1789882691.632364 = 5.156 s
```

RTO observado: 4,6–6,5 s; mediana 5,2 s. Siempre es una cota superior: la sonda con `statement_timeout = 2 s` solo ve la recuperación cuando termina un intento (resolución $\approx$ un intento). La variación entre corridas cabe en esa resolución, más la diferencia de hardware y de cuándo expira la *liveness* del nodo caído respecto al último latido. El orden de magnitud es estable. El RTO no depende de un operador: lo marca el tiempo en que el clúster declara muerto al nodo y otro votante toma el lease.

En la corrida 3, 12 escrituras posteriores al stop tardaron más de 100 ms (máx. 1,47 s). El lease se recuperó, pero el rango siguió funcionando sin un votante hasta el `docker start`, en una máquina ocupada. Ninguna de esas escrituras falló.

RPO: ninguna escritura confirmada se perdió

`p1/chaos/rpo.py` convierte el RPO en una comprobación aritmética. La sonda pide folios consecutivos y guarda el folio que recibió cada intento OK. Si el clúster "olvidara" una escritura confirmada, un folio se repetiría o retrocedería. Tres pruebas por corrida (`-rpo.txt`):

Prueba	Corrida 1	Corrida 2	Corrida 3
1. Folios confirmados sin repetir ni retroceder	OK (2 → 279)	OK (2 → 268)	OK (270 → 543)
2. Primer OK tras el stop = último antes + 1	30 = 29 + 1	30 = 29 + 1	298 = 297 + 1
3. Folio final en la base = último confirmado	279 = 279	268 = 268	543 = 543
RPO para lo confirmado	0	0	0

Qué significa y qué no. RPO = 0 se refiere a las escrituras **confirmadas** al cliente. Es la garantía de Raft: un commit se confirma solo cuando la entrada está persistida en la mayoría (2 de 3). Cualquier mayoría futura intersecta con esa, así que el nuevo leaseholder tiene la escritura. Los intentos con timeout (2, 3 y 2) tienen

resultado **desconocido** para el cliente. La prueba 2 lo resuelve: si alguno se hubiera confirmado en el servidor, el folio siguiente saltaría (29 → 31). No saltó en ninguna corrida, así que ninguno se confirmó. Una aplicación real tendría que reintentar esos intentos de forma idempotente, porque desde el cliente no se distingue "no se hizo" de "se hizo y no me enteré".

Caída de una región (opcional) y efecto en sus filas RBR

Script: `p1/chaos/region.sh` (orquestra) y `p1/chaos/region.py` (sonda, leases y resumen). Evidencia: `evidence/p1/e4-region-cr-limon-*`. Corrida: 2026-09-20, chris-HP (4 CPU / 11,8 GiB).

Qué cambia respecto a la prueba anterior. Con un nodo por región, detener `crdb-2` es perder `cr-limon` completa. Aquí **no se mueve ningún lease**: se observa la colocación diseñada en E1 §8 (cada fragmento con su leaseholder en casa) y se mide cada fragmento por separado. Hay seis casos, cada uno en su hilo y su conexión a `crdb-1`, cada 0,2 s y con `statement_timeout = 2 s`. Así, un caso bloqueado no retrasa a los demás. Las escrituras son las transferencias de E3 (doble asiento), y `check.py` dio 10/10 después de la prueba (`-despues.txt`).

Quién atiende cada fragmento (`-antes.txt`, `-durante.txt`, `-despues.txt`):

Momento	cuenta_sj	cuenta_limon	moneda
Antes	cr-sj	cr-limon (hogar)	cr-sj (coordina escrituras; cada región lee su copia)
15 s después del stop	cr-sj	us-east (fuera de hogar)	cr-sj
Tras <code>docker start</code>	cr-sj	cr-limon, de vuelta en 30 s	cr-sj

Resultado por caso (`-resumen.txt`; región caída 31,8 s):

Caso	p50 antes (ms)	Errores durante	Vuelve a responder (s)	Estable desde (s)	p50 estable (ms)	Errores al reintegrarse
lee-limon (O1, región caída)	1,61	3	5,9	10,0	1,56	2
escribe-limon (O4, región caída)	26,75	4	6,3	11,8	24,14	2

Caso	p50 antes (ms)	Errores durante	Vuelve a responder (s)	Estable desde (s)	p50 estable (ms)	Errores al reintegrarse
escribe-cruza (O5, cr-sj ↔ cr-limon)	54,29	4	9,5	12,4	23,79	2
lee-sj (O1, región viva)	1,10	2	0,04	12,0	14,16	2
escribe-sj (O4, región viva)	23,10	2	0,07	12,2	23,16	2
lee-monedas (O7, GLOBAL)	1,07	0	0,04	0	0,95	1

Interpretación.

1. **Las filas de la región caída no se pierden ni quedan inaccesibles: quedan ~6 s sin servicio y después las atiende otra región.** Sus votantes Raft están repartidos uno por región (E1 §7–§8), así que al caer cr-limon cada rango *_limon conserva 2 de 3 votantes. El lease pasa a us-east y las operaciones O1/O4 sobre cuentas de cr-limon vuelven a los ~6 s, lo mismo que el RTO de la prueba anterior. Durante la caída leer una cuenta de cr-limon cuesta lo mismo que antes (1,56 frente a 1,61 ms): sin latencia inyectada, ir a us-east o a cr-limon cuesta igual desde cr-sj.
2. **Esa disponibilidad es la otra cara del límite de residencia.** Las filas de cr-limon sobreviven a la pérdida de su región *porque* tienen copias fuera de ella. Con la topología que exige la residencia (3 nodos por región + PLACEMENT RESTRICTED, E1 §4), los 3 votantes de cliente_limon estarían en cr-limon. Perder la región dejaría sus filas **sin servicio** hasta que vuelva, mientras que cr-sj y us-east seguirían intactas. Residencia estricta y supervivencia a la pérdida de región son incompatibles para un mismo fragmento. Para tener las dos, CockroachDB ofrece SURVIVE REGION FAILURE, que coloca réplicas en otras regiones, es decir, renuncia a la residencia estricta.
3. **monedas (GLOBAL) no se interrumpió** durante la caída: cero errores y la misma latencia. Cada región lee su propia copia (E1 §6). Es la tabla que justifica su replicación completa.
4. **La transferencia que cruza regiones es la última en volver (9,5 s)**, porque necesita que estén disponibles los rangos de las dos regiones que toca. Con la región caída, su p50 bajó de 54 a 24 ms: ya no espera al leaseholder en crdb-2, sino a uno en us-east, en una máquina con un nodo menos compitiendo por CPU.
5. **Las regiones vivas también notaron la falla.** lee-sj y escribe-sj tuvieron 2 timeouts cada uno entre los 6 y 12 s, y lee-sj quedó en ~14 ms hasta la reintegración. Hay dos causas plausibles: el clúster reubica los leases de rangos de sistema que estaban en crdb-2, y cada rango *_sj queda con 2 votantes, así que cada escritura necesita a los dos. El aumento de lee-sj no se reproduce aislado: con crdb-2 detenido, una lectura cr-sj sin escritores o con uno sigue en ~1 ms (e4-region-diagnostico.txt). Aparece solo con la caída más los seis casos concurrentes, que comparten filas (escribe-sj y escribe-cruza escriben la misma cuenta). Se declara como observado, no como explicado del todo.
6. **Volver a tres nodos tampoco es gratis.** Entre 13 y 21 s después del docker start, **todos** los casos,

incluso moneda, tuvieron 1–2 timeouts: el nodo reintegrado se pone al día (Raft) y los leases de `cr - limon` vuelven a su región por `lease_preferences`, lo que tardó 30 s. Una reintegración real conviene hacerla fuera de horas pico.

Límites. Una sola corrida de la caída de región, porque la tarea es opcional. Con un nodo por región, "región" y "nodo" coinciden, y las regiones son lógicas, sin latencia inyectada.

E5 — Crítica: ¿hacía falta distribuir? (15 %)

Baseline: p1/baseline/ (mismo esquema traducido, mismos datos —huella ad995e29a16440f7—, mismas cuatro operaciones de p1/bench/latency.py, mismo método y aislamiento SERIALIZABLE). Evidencia: evidence/p1/e5-latency-postgres-summary.txt, e5-latency-postgres.csv, e5-docker-stats.txt. La comparación se hace sobre la **misma máquina** (12 CPU / 7,9 GiB): CockroachDB en evidence/p1/otra-maquina/e3-latency-summary.txt, PostgreSQL en e5-latency-postgres-summary.txt.

Latencia (n = 100 por caso, warm-up 10, ms)

Operación	CockroachDB ×3		PostgreSQL ×1		Razón	
	p50	p99	p50	p99	p50	p99
Lectura local (O1, fila cr-sj)	0.770	6.458	0.171	0.250	4.5×	26×
Lectura remota (O1, fila cr-limon)	1.520	17.342	0.172	0.348	8.8×	50×
Escritura local (O4)	16.467	31.236	1.400	1.700	11.8×	18×
Escritura que cruza región (O5)	15.073	37.296	1.426	1.785	10.6×	21×

Lectura:

- En PostgreSQL local = remota (0,17 ms): la región es un dato de la fila; no hay gateway ni leaseholder al que ir. En CockroachDB la lectura remota paga el salto al leaseholder de cr-limon, y su cola (p99 17 ms, 17/100 muestras > 10 ms) es 50× la de PostgreSQL.
- La escritura en CockroachDB cuesta ~11× la de un nodo. Es el precio de replicar en 3 votantes y esperar la mayoría 2/3 antes de confirmar (E3 §2); en PostgreSQL el COMMIT solo espera el fsync local del WAL (~0,7 ms medido con synchronous_commit = on).
- Las dos escrituras de CockroachDB cuestan lo mismo entre sí por la topología de un nodo por región (E3 §2); en PostgreSQL cuestan lo mismo porque no hay distribución. El motivo es distinto.

Costo en la misma máquina (e5-docker-stats.txt)

Recurso	CockroachDB ×3	PostgreSQL ×1	Razón
Memoria en reposo	634 + 605 + 624 ≈ 1 863 MiB	88 MiB	21×
CPU bajo carga (8 000 operaciones, pico)	188 % + 154 % + 132 % ≈ 474 %	42 %	11×

Recurso	CockroachDB ×3	PostgreSQL ×1	Razón
Tiempo para esas 8 000 operaciones	≈ 130 s (66 muestras de 2 s)	≈ 10 s (5 muestras)	13×
Disco (volúmenes; ambos incluyen la base de datos del curso)	3 × 1,27 GB ≈ 3,8 GB	134 MB	28×
Tiempo de carga del seed (97 527 filas)	36,1 s	6,3 s	5.7×

Complejidad operativa

	CockroachDB ×3	PostgreSQL ×1
Piezas	3 contenedores + 1 de <code>init</code> , 3 volúmenes, red	1 contenedor, 1 volumen
Configuración	licencia, región primaria + 2 regiones, <code>LOCALITY</code> por tabla, <code>SURVIVE ZONE FAILURE</code> , zona RF=3 para E4; 7 pasos en <code>setup.sh</code>	un <code>CREATE DATABASE</code> y el esquema; 2 comandos
Cosas que hay que saber para operar	<code>node_id</code> ≠ número de contenedor, leaseholders, subreplicación RBR con un nodo por región, gracia de licencia	<code>autovacuum/checkpoint</code> tras cargas masivas
Reproducción en otra máquina	verificada (E2)	verificada (este baseline)

Lo que esta comparación no dice sola

Los números anteriores miden lo que **cuesta** distribuir. Lo que **compra** está en otros entregables y es lo que la conclusión de E5 tiene que poner en la balanza: sobrevivir a la caída de un nodo confirmando escrituras con 2 de 3 y RPO 0 para lo confirmado (E4), región hogar y leaseholder por región para la PII (E1 §4, con su límite declarado), y `SERIALIZABLE` sin pedirlo. PostgreSQL de un nodo no tiene ninguna de las tres; con una réplica de lectura gana lecturas escalables y un respaldo, pero no confirmación con mayoría ni RPO 0 ante la pérdida del primario.

La alternativa del enunciado: 1 nodo primario + réplica de lectura

El enunciado pide comparar contra "1 primario + réplica de lectura". Esa arquitectura **no se montó**: con replicación asíncrona la réplica no cambia la latencia de escritura del primario, que es lo que mide `latency_pg.py`, y las lecturas de este dominio ya se sirven en 0,17 ms. Lo que sigue es su descripción y qué pasaría con cada número medido.

Cómo sería. PostgreSQL 16 con *streaming replication*: un primario que acepta lecturas y escrituras y una o más

réplicas en modo *hot standby*, de solo lectura, que reciben el WAL y lo aplican. La aplicación necesita separar el tráfico: escrituras y lecturas que deban ver su propia escritura van al primario; las demás (O1, O2, O9) pueden ir a la réplica. En la práctica se pone un *pooler* (PgBouncer, Pgpool-II) o se resuelve en el cliente.

	Réplica asíncrona (<code>synchronous_commit = off/local</code>)	Réplica síncrona (<code>remote_apply / on</code>)
Latencia de escritura	La del nodo único (1,4 ms medidos): el primario no espera a la réplica	El COMMIT espera a la réplica; con la réplica en la misma máquina, ~2–3 ms; entre regiones, el RTT
RPO si se pierde el primario	> 0: se pierden las transacciones confirmadas que no alcanzaron a enviarse	0 para lo confirmado, igual que el clúster
RTO	Promoción manual o de una herramienta externa (Patroni, repmgr): minutos	Igual: la promoción sigue siendo externa
Si cae la réplica	El primario sigue	Con <code>synchronous_standby_names</code> estricto, el primario se bloquea hasta que vuelva

Qué compra. (1) Lecturas escalables: los reportes (O9) y las consultas de saldo se reparten entre réplicas sin tocar el primario. (2) Una copia para recuperarse. (3) Casi toda la latencia del nodo único: las lecturas en la réplica valdrían ~0,17 ms, frente a 0,77–1,52 ms en el clúster. (4) Operación mucho más simple que el clúster: dos contenedores y un archivo de configuración, sin licencia, regiones ni `LOCALITY`.

Qué no compra, y por qué importa aquí.

1. **No cumple la residencia; la empeora.** La réplica recibe el WAL **completo**: toda la PII de las tres regiones queda copiada en un segundo sitio. Si la réplica se pone en otra región para servir lecturas cerca, los nombres y documentos de los clientes de `cr-sj` salen de su región. No hay forma de replicar "solo las filas de esta región" con replicación física; habría que pasar a replicación lógica por publicación filtrada, con un primario por región, que es reconstruir a mano la fragmentación que `REGIONAL BY ROW` da declarada (E1 §3).
2. **La conmutación no es automática.** En el clúster, la caída del leaseholder se resolvió sola en 4,6–6,5 s, tres veces de tres (E4). Promover una réplica exige decidir y ejecutar, o montar otra pieza (Patroni + etcd) que vuelve a acercar la complejidad operativa a la del clúster.
3. **Con réplica asíncrona, el RPO no es 0.** Es la diferencia de fondo: el clúster confirma con la mayoría 2 de 3 *antes* de responder al cliente, y por eso ninguna escritura confirmada se perdió en las tres corridas de E4. La réplica asíncrona responde antes de que el dato salga del primario.
4. **La escritura no escala ni se acerca al usuario.** Sigue habiendo un solo primario: todas las escrituras de las tres regiones viajan hasta él.

Si el requisito fuera solo disponibilidad, la mejor opción no es esta ni el clúster multi-región, sino **primario + réplica síncrona en la misma región con conmutación automática**: RPO 0, RTO de segundos a pocos minutos, latencias cercanas a las del nodo único y ninguna región que configurar. Es la alternativa que la conclusión pone

en la balanza.

Conclusión: justificado solo por residencia

Veredicto. Para este dominio y este volumen, distribuir en tres regiones **no se justifica por rendimiento ni por costo**. Tampoco lo justifica la disponibilidad sola. **Se justifica solo por la regla de residencia:** la PII de cada cliente debe vivir en su región de apertura, y ningún diseño de un solo primario puede cumplirla. Además, el montaje de este proyecto (un nodo por región) todavía no la cumple del todo.

1. Latencia: el nodo único gana en todas las operaciones. En la misma máquina, PostgreSQL lee un saldo en 0,17 ms y CockroachDB en 0,77 ms (local) o 1,52 ms (remota): entre 4,5× y 8,8× más lento en la mediana, y entre 26× y 50× en el p99. Una transferencia cuesta 1,4 ms en un nodo y 15–16 ms en el clúster, ~11×. Nada de eso es un defecto de configuración: es el precio de que cada commit espere a una mayoría de 2 de 3 réplicas (E3 §2). Con 97 527 filas y un día de operaciones que un solo servidor atiende en segundos, no hay carga que repartir. Distribuir no acerca los datos a ningún usuario que hoy esté lejos, porque las regiones son lógicas y no hay WAN entre ellas.

2. Costo y operación: el clúster cuesta un orden de magnitud más. En reposo usa 21× la memoria ($\approx 1\,863$ MiB frente a 88 MiB) y 28× el disco. Bajo la misma carga de 8 000 operaciones usa 11× la CPU y tarda 13× más. Operar exige saber cosas que un nodo no pide: la licencia, las regiones, que `node_id` no es el número del contenedor, dónde está cada leaseholder y por qué la configuración RBR declarada no coincide con la observada. Todas quedaron documentadas en E1 §4, E3 §4 y E4.

3. Disponibilidad: el clúster sí compra algo, pero no lo suficiente para justificarlo solo. Al detener el nodo que tenía el lease, el clúster volvió a confirmar escrituras en **4,6 s** (cota superior de la sonda; el real está entre ~3,9 y 4,6 s), **sin intervención humana**. Siguió atendiendo con 2 de 3 nodos durante los 15 s de la falla (E4, corrida 1). Lo repitió tres veces, con RTO de 4,6 / 6,5 / 5,2 s, y en las tres el RPO para escrituras confirmadas fue **0**: los folios confirmados no se repiten, no retroceden y el primero después del stop es el último antes + 1 (E4, `-rpo.txt`). La alternativa de **1 primario + réplica de lectura** no ofrece eso. Si la réplica es asíncrona, perder el primario puede perder transacciones ya confirmadas ($RPO > 0$). Además, promoverla es una decisión manual o de una herramienta externa, con un RTO de minutos. La réplica sí le da lecturas escalables y una copia para recuperarse. Aun así, si el único requisito fuera sobrevivir a la caída de un servidor, habría un camino mucho más barato que el multi-región: un primario con una réplica **síncrona** en la misma región y conmutación automática. Tendría RPO 0 y un RTO de segundos a pocos minutos, conservaría las latencias del nodo único para las lecturas (el commit solo esperaría a una réplica cercana) y no necesitaría regiones, licencia ni `LOCALITY`. La disponibilidad pide replicar, no distribuir.

4. Residencia: el único requisito que un primario único no puede cumplir. Con un solo primario, toda la PII vive donde esté ese servidor. Si está en San José, los nombres y documentos de los clientes de `us-east` salen de su región de apertura. Ponerlo en otra región solo cambia a quién le toca. Una réplica de lectura lo empeora, porque copia **toda** la PII a un segundo sitio. La residencia exige que el dato de cada cliente viva en una región distinta según su fila, y eso es fragmentación horizontal por `region` con asignación por región (E1 §3–§6). Es justo lo que ofrece `REGIONAL BY ROW`, y ningún motor de un solo nodo lo hace.

El límite que hay que decir en la defensa. Este montaje demuestra la mitad de la residencia: la región hogar de cada fila y su leaseholder están en la región de apertura (E2, E3). Pero con **un nodo por región**, cada rango RBR tiene votantes en las tres regiones, así que hoy existen copias físicas de la PII fuera de su región (E1 §4). Para cumplir la regla del todo hacen falta al menos 3 nodos por región (9 en total) y `PLACEMENT RESTRICTED`, de modo que los 3 votantes de cada partición quepan en su región hogar. El costo medido aquí, de ~20× en memoria, se multiplicaría otra vez por 3.

En resumen: si el banco no tuviera clientes cuya PII deba quedarse en otra jurisdicción, lo correcto sería **no distribuir**. Bastaría con PostgreSQL de un nodo con réplica síncrona para la disponibilidad y una réplica de lectura si las lecturas crecieran. Como la regla de residencia es parte del enunciado, la distribución por región está **justificada, pero solo por ella**, y solo si se despliega con los nodos por región que la hacen efectiva. Ni la latencia, ni el costo, ni la disponibilidad medidos en este proyecto la justifican por sí solos.

Apéndice A — Scripts del proyecto

Todo lo que produce un número en este informe lo genera un script versionado. Ninguna cifra se escribió a mano.

Archivo	Qué hace	Qué genera
<code>p1/setup.sh</code>	Levanta el clúster, licencia, regiones, esquema, datos y verificación (7 pasos, idempotente)	—
<code>p1/check.py</code>	Verificador de solo lectura: 10 comprobaciones de clúster, esquema, fragmentación y datos	<code>e2-check.txt</code>
<code>p1/evidence.sh</code>	Evidencia de E1 y E2	<code>e2-node-status.txt</code> , <code>e2-inspect.txt</code> , <code>e2-check.txt</code> , <code>e1-verificacion.txt</code> , <code>e1-asignacion.txt</code>
<code>p1/sql/00_database.sql</code>	Base <code>p1_banca</code> , región primaria y regiones	—
<code>p1/sql/01_schema.sql</code>	Esquema del dominio con <code>LOCALITY</code> (<code>GLOBAL</code> y <code>REGIONAL BY ROW</code>)	—
<code>p1/sql/02_e4_table.sql</code>	Tabla de folios en base sin regiones con <code>RF=3</code> (E4)	—
<code>p1/sql/inspect.sql</code>	<code>SHOW REGIONS</code> , <code>SHOW CREATE</code> , zonas y <code>SHOW RANGES</code>	<code>e2-inspect.txt</code>
<code>p1/sql/verificacion.sql</code>	Compleitud, reconstrucción, disyunción, derivada y doble asiento	<code>e1-verificacion.txt</code>
<code>p1/sql/asignacion.sql</code>	Fragmento → rango → leaseholder y votantes (asignación observada)	<code>e1-asignacion.txt</code>
<code>p1/gen/seed.py</code>	Generador determinista (semilla 4601) con huella de datos	—

Archivo	Qué hace	Qué genera
p1/bench/latency.py	Benchmark de E3: 4 casos, p50/p99, CSV crudo	e3-latency.csv, e3-latency-summary.txt
p1/chaos/falla.sh	Falla de un nodo: escritura sana → lease → stop → sonda → start → RTO	e4-corrida<N>-*
p1/chaos/sonda.py	Sonda de escrituras (folios) cada 0,2 s con statement_timeout de 2 s	e4-corrida<N>.csv
p1/chaos/rto.py	RTO desde el CSV y la época del stop, mostrando la resta	e4-corrida<N>-rto.txt
p1/chaos/rpo.py	RPO: secuencia de folios, cruce de la falla y folio final	e4-corrida<N>-rpo.txt
p1/chaos/region.sh · region.py	Caída de una región completa; 6 casos en paralelo	e4-region-<region>-*
p1/baseline/seed_pg.py	Mismos datos en PostgreSQL (misma huella)	—
p1/baseline/latency_pg.py	Mismo benchmark contra PostgreSQL de un nodo	e5-latency-postgres*
p1/baseline/docker_stats.sh	Costo comparado: CPU, memoria, disco y piezas a operar	e5-docker-stats.txt
p1/informe.sh	Arma este informe y comprueba que no contenga la licencia	entregas/p1/INFORME.md y .pdf

Apéndice B — Evidencia

Todos los archivos están en `evidence/p1/` y llevan marca de tiempo y nombre de host.

Grupo	Archivos	Entregable
Clúster y esquema	<code>e2-node-status.txt</code> , <code>e2-inspect.txt</code> , <code>e2-check.txt</code>	E2
Fragmentación	<code>e1-verificacion.txt</code> , <code>e1-asignacion.txt</code>	E1
Latencia	<code>e3-latency.csv</code> , <code>e3-latency-summary.txt</code>	E3
Reproducción en otra máquina	<code>otra-maquina/e2-check.txt</code> , <code>otra-maquina/e2-node-status.txt</code> , <code>otra-maquina/e3-latency-summary.txt</code>	E2, E3
Falla de nodo (3 corridas)	<code>e4-corrida{1,2,3}{-antes.txt, .csv, -sonda.txt, -stop.epoch, -stop.txt, -despues.txt, -rto.txt, -rpo.txt}</code>	E4
Caída de región	<code>e4-region-cr-limon-*</code> , <code>e4-region-diagnostico.txt</code>	E4
PostgreSQL y costo	<code>e5-latency-postgres.csv</code> , <code>e5-latency-postgres-summary.txt</code> , <code>e5-docker-stats.txt</code>	E5

Reproducir desde cero en una máquina limpia: `p1/setup.sh`, luego `p1/evidence.sh`, el benchmark de E3, `p1/chaos/falla.sh` y el baseline de E5. Los comandos exactos están en el entregable E2 de este informe (`p1/README.md`).

Apéndice C — Roles del equipo

(Completar el rol y los entregables de cada integrante antes de entregar: la rúbrica evalúa participación individual y la contrasta con los commits. `git short log -sne` lista los commits por integrante.)

Integrante	Carné	Rol principal	Entregables en los que trabajó	Qué debe poder defender
Christopher Rodriguez C.	2022040771			
Fabricio Mena M.	2019042722			
Alisson Redondo M.	2021510425			

Apéndice D — Límites declarados

Se listan juntos porque son la parte del trabajo que más se pregunta en la defensa.

1. **Las regiones son lógicas.** Tres contenedores en una máquina, sin latencia inyectada. Ninguna cifra de E3 representa una WAN.
 2. **Las particiones REGIONAL BY ROW no cumplen su configuración de zona.** Piden 3 votantes dentro de su región hogar; con un nodo por región, los 3 votantes quedan repartidos entre las tres. Por eso la escritura "local" también espera a otra región (E3) y por eso la residencia física no se cumple (E1 §4).
 3. **La prueba de falla usa una tabla en una base sin regiones** con RF=3, que sí cumple lo que declara, para que la mayoría 2/3 medida sea la declarada.
 4. **El RTO medido es una cota superior**, con resolución de un intento de la sonda (0,2 s más el `statement_timeout` de 2 s).
 5. **La caída de región se corrió una vez** (era opcional) y, con un nodo por región, coincide con la caída de un nodo.
 6. **E5 compara contra PostgreSQL con configuración por defecto**, sin los límites de memoria de los nodos CockroachDB; la réplica de lectura de la alternativa se describe pero no se montó.
 7. **El volumen de PostgreSQL incluye la base ti4601 de los labs**, así que su tamaño en disco es una cota superior.
-